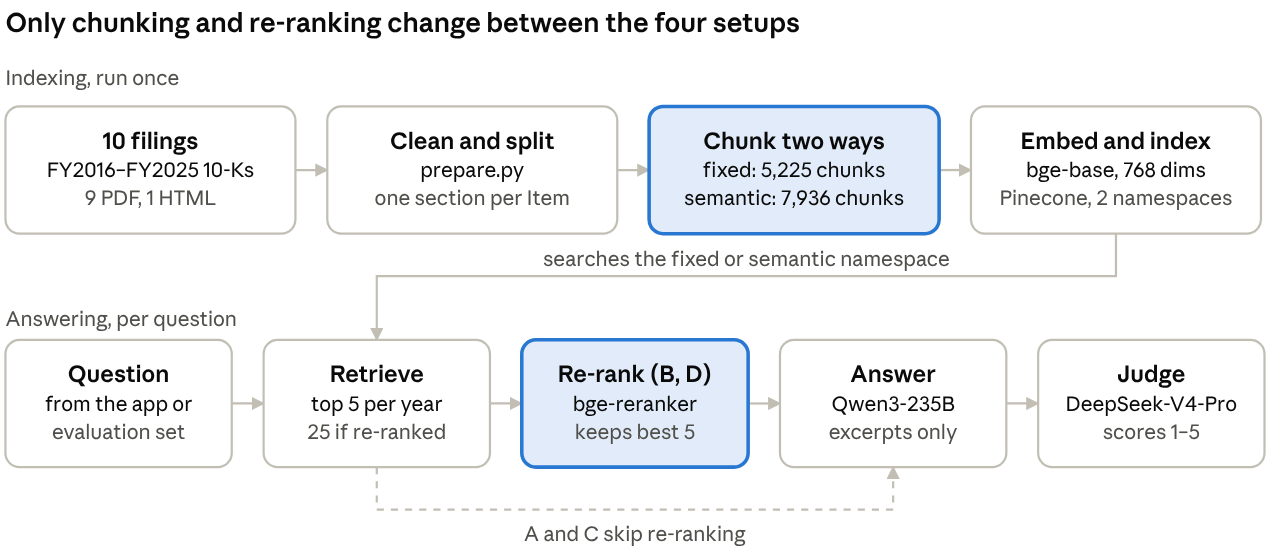
Tesla 10-K RAG: Chunking and Re-ranking Report

Oct 25, 2026 · @Surya

Fixed-size chunking without re-ranking (setup A) answered Tesla 10-K questions best, scoring 4.25 out of 5 on 67 questions. Re-ranking hurt fixed chunks but helped semantic chunks, and everyday wording cut every setup's score by 1.2 to 1.5 points, mostly because the right passage was not retrieved.

Pipeline

The pipeline runs end to end: Tesla's 10-K filings for FY2016–FY2025 are cleaned, chunked two ways, indexed in Pinecone and answered through a local web app or the evaluation script.



pipeline · indexing once, then answering per question

Setups A and C send the retrieved chunks straight to the answer model. B and D first fetch 25 chunks per year and keep the re-ranker's best 5.

Component	Choice
Embeddings	BAAI/bge-base-en-v1.5, 768 dimensions, run locally

Component	Choice
Vector store	Pinecone serverless, index `tenk-rag`, one namespace per chunking strategy
Re-ranker	BAAI/bge-reranker-base, run locally
Answer model	Qwen3-235B-A22B-Instruct-2507 on Nebius Token Factory
Judge model	DeepSeek-V4-Pro-0813 on Nebius Token Factory
Interface	Streamlit web page (`app.py`): pick a setup or compare all four, see every retrieved chunk

Evaluation design

Four setups were compared on the same questions. Only chunking and re-ranking changed: embeddings, prompt, answer model and judge stayed the same.

Setup	Chunking	Re-ranking
A	Fixed: 1,000 characters, 200 overlap	No
B	Fixed: 1,000 characters, 200 overlap	Yes: top 25 by similarity, re-scored, best 5 kept
C	Semantic: split at 90th-percentile meaning shifts, capped at 1,500 characters	No
D	Semantic: split at 90th-percentile meaning shifts, capped at 1,500 characters	Yes: top 25 by similarity, re-scored, best 5 kept

- **Questions:** 67, each with a reference answer checked against the filings. 45 use the filings' own terms (17 lookups, 2 unit conversions, 11 multi-year, 12 narrative, 3 unanswerable). 22 use everyday wording (9 lookups, 6 definitions, 5 reasoning, 1 narrative, 1 multi-year); 3 of these came from testing the app.
- **Retrieval:** 5 chunks per fiscal year the question covers, so a two-year question gets 10.

- **Correctness and faithfulness (1–5):** scored by a judge model (DeepSeek-V4-Pro) that differs from the answer model (Qwen3-235B-A22B-Instruct). Correctness compares the answer with the reference; faithfulness checks every claim against the retrieved chunks.
- **Fact recall and context recall (0–1):** the share of key figures found in the answer, and in the retrieved chunks.

Chunking strategy comparison

Without re-ranking, fixed chunks beat semantic chunks: A scored higher than C on 11 questions, lower on 3, and tied on 53. With re-ranking the two were close, and D edged B 10 to 8.

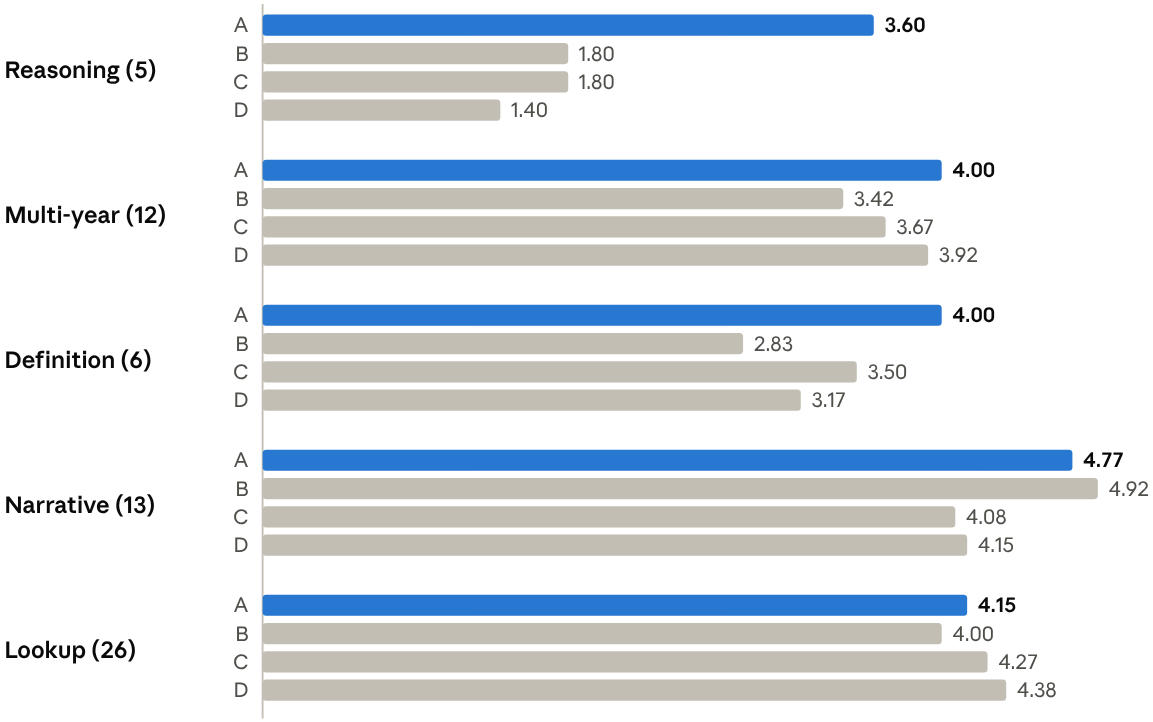
Setup	Correctness, filing wording (45)	Correctness, everyday wording (22)	Correctness, all (67)	Faithfulness, all	Context recall, all
A: fixed	4.67	3.41	4.25	4.75	0.78
D: semantic + re-rank	4.36	3.18	3.97	4.42	0.77
C: semantic	4.36	3.05	3.93	4.64	0.73
B: fixed + re-rank	4.38	2.86	3.88	4.75	0.76

- **Everyday wording is the bigger effect.** Every setup lost 1.2 to 1.5 points on casual questions, and the share of key facts retrieved fell from 0.80–0.86 to 0.58–0.66.
- **Semantic chunks are weakest on narrative questions** (A 4.77, C 4.08). In the cleaned text, 76% of lines end mid-sentence where the PDF wrapped, and the semantic splitter breaks on them, so chunks often stop mid-sentence ("…and 4) post-incident").
- **Semantic chunks match reworded questions better.** For "how many cars did tesla sell in 2024?", the chunk with the deliveries figure ranked 8th among fixed chunks, outside the top 5, but 4th among semantic chunks.
- **Fixed chunks make the model more cautious.** A scored 5.0 on faithfulness for everyday questions, against 4.3 for D: when A lacked the figure it said so, while D more often answered from loosely related text.

A leads on reasoning, multi-year and definition questions; B and D lead elsewhere

Mean correctness (1 to 5) by question type, bars from 0

A fixed · B fixed + re-rank · C semantic · D semantic + re-rank



Left out: unit-conversion (2) and unanswerable (3) questions, where every setup scored 5.00.

results/detail.csv · 67 questions, mean judge correctness per setup and question type

The widest gap is on the five reasoning questions, which combine several figures: A answered 3 of them fully correctly, B answered 1, and C and D none. On one of them, B, C and D said Tesla hired more people in 2024, though headcount fell from 140,473 to 125,665.

Re-ranking impact

Re-ranking made fixed chunks worse and semantic chunks slightly better. Its clearest wins were questions worded differently from the filing, where the right chunk ranked just outside the top 5 by similarity.

Comparison (per-question correctness)	All 67	Filing wording (45)	Everyday wording (22)
Fixed: A (no re-rank) vs B (re-rank)	A 13, B 6, 48 ties	A 6, B 3, 36 ties	A 7, B 3, 12 ties

Comparison (per-question correctness)	All 67	Filing wording (45)	Everyday wording (22)
Semantic: C (no re-rank) vs D (re-rank)	C 6, D 8, 53 ties	C 3, D 3, 39 ties	C 3, D 5, 14 ties

Where it hurt. The re-ranker (bge-reranker-base) tended to push out chunks that are mostly rows of numbers. For FY2023 total revenue, FY2024 automotive revenue and revenue in 2019 against 2023, A answered correctly while B replied that the filings did not contain the figure.

Where it helped.

- *"How many cars did tesla sell in 2024?"* A missed the deliveries chunk (ranked 8th). B fetched 25 candidates and the re-ranker moved it into the top 5, so B answered about 1,789,000.
- *"Between 2016 and 2024 how many total vehicles did tesla deliver?"* B and D found the deliveries sentence in all nine filings and added the yearly figures correctly to 7,114,347. A missed 2022 and C missed 2021, so both declined to give a total. The filing-backed answer is 7,063,647 for 2017–2024: the 2016 figure B and D used (about 50,700) counts only deliveries recognised in automotive revenue, and the filings give no 2016 total.

Per-year retrieval mattered more than re-ranking for multi-year questions. Pooling all years and keeping the top 5 let one year crowd out the others. Giving each year its own 5 chunks raised correctness on the 11 filing-worded multi-year questions for every setup:

Setup	Shared top 5	Top 5 per year
A: fixed	3.73	4.27
B: fixed + re-rank	2.82	3.64
C: semantic	3.18	3.91
D: semantic + re-rank	2.91	3.91

Limitations

The ranking of A first is consistent across both question sets, but the gaps between B, C and D are within run-to-run noise.

- **Semantic chunking was not tested at its best.** The PDF line-wrap problem was found but not fixed; joining wrapped lines before chunking is the fair re-test.
- **Sample size and variance.** Each question ran once per setup. On the 22 everyday questions, a 0.2-point gap is about 4 questions, and the same question can get noticeably different answers on another run.
- **The prompt and judge are strict.** The prompt tells the model to refuse the whole question when any part is missing, so it skips partial answers. The judge marked correct computed differences (for example a headcount change) as unfaithful because the result is not printed in the excerpts.
- **One reference was too narrow.** For "how much cash did tesla have sitting around at the end of 2023?", every setup answered $16,398 million (cash and cash equivalents); the reference used $29.09 billion including investments.
- **FY2025 has no financial data.** The corpus holds Tesla's FY2025 10-K/A, which covers only Part III (Items 10–14). The original FY2025 10-K with the financial statements is not included.
- **The question set was written by the project author,** including the everyday set, which was written after three app test questions had been seen. Those three are tagged so results can be read with and without them (A 3.58, D 3.26, C 3.16, B 2.95 without them).
- **The app differs from the evaluation in one way:** the evaluation fixes each question's years, while the app reads years from the question and searches all filings when none is named.

Reproduce

The project lives in `C:\AgenticAI-Projects\rag10k`; keys go in `.env` (Pinecone, plus Nebius or Gemini for the language models).

1. `python prepare.py`: extract, clean and split the 10 filings into 10-K Items (`data/clean/`).
2. `python make_chunks.py`: build both chunk sets (`data/chunks/fixed.jsonl`, 5,225 chunks; `semantic.jsonl`, 7,936 chunks). Semantic chunking takes about 2 hours on a CPU.
3. `python index.py`: embed and upload to Pinecone index `tenk-rag`, namespaces `fixed` and `semantic`.

4. `python evaluate.py`: answer and judge all questions in `questions.json` for setups A–D; results cache in `results/cache.jsonl`, scores in `results/summary.csv` and `results/summary_by_wording.csv`.
5. `streamlit run app.py --server.address localhost`: the question-answering web page, with a setup picker, a side-by-side comparison of all four setups, and the retrieved chunks for every answer.